



Database Management Systems

Triggers

Dr. Nagasundari S

Department of Computer Science and Engineering

Database Management Systems

Unit 2: Advanced SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Dr. Nagasundari S

Department of Computer Science and Engineering

- A trigger is a statement that the system executes automatically as a side effect of a modification to the database.

Defining a trigger involves:

- Specifying when a trigger needs to be executed. This consists of two components:
 - Event** – which causes the trigger to be checked
 - Condition** – which must be satisfied for trigger execution to proceed
- Specifying the **action** to be taken when the trigger executes
- Once we enter a trigger into the database, the system executes it whenever the specified event occurs and the corresponding condition is satisfied.

- **Triggers can be used to implement certain integrity constraints that cannot be specified using the constraint mechanism of SQL.**
- Triggers are also useful mechanisms for alerting humans or for starting certain tasks automatically when certain conditions are met.
- Consider a scenario wherein a student has taken up a new course and a new tuple needs to be inserted in the *TAKES* relation. We could design a trigger that updates the tuple in the *STUDENT* relation for the student taking the course by adding the number of credits for the course to the student's total credits, whenever a new tuple is inserted in the *TAKES* relation.
- Triggers usually cannot perform updates outside the database. However, some database systems provide built-in support for sending email from triggers.
- Triggers can be used in various applications, such as maintaining database consistency, monitoring database updates, and updating derived data automatically.

Let's have a look at the syntax of the CREATE TRIGGER statement.

Syntax:

```
CREATE TRIGGER trigger_name
(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name
FOR EACH ROW
BEGIN
    --variable declarations
    --trigger code
END;
```

- The CREATE TRIGGER statement is used to create a trigger with the name *trigger_name* on the relation *table_name*.

Let's try to understand the syntax of the CREATE TRIGGER statement:

CREATE TRIGGER trigger_name

- The first line of the syntax (shown above) is used to **specify the name of the trigger.**

(AFTER | BEFORE) (INSERT | UPDATE | DELETE) ON table_name

- The second line of the syntax is used to specify the **event which causes the trigger to be checked.**

- table_name - specifies the **relation** which the trigger must check for modification
- INSERT | UPDATE | DELETE - specifies the **type of modification to be checked** i.e. whether the trigger must be checked on insertion, updation or deletion of tuples.
- AFTER | BEFORE - specifies **when the trigger must be checked** i.e. before or after the specified relation undergoes modification.

FOR EACH ROW

- Multiple tuples could be modified by a single SQL statement(for eg, multiple tuples could be inserted using one INSERT statement). The third line specifies that the trigger code would explicitly iterate over each modified row.
- The part of the query between the **BEGIN** and **END** keywords corresponds to the **action** to be taken on the modified tuples. This involves declaring any variables that would be used and specifying the SQL statements to be executed. Actions could be the execution of stored procedures as well.

Consider a sample database that stores information about students and the marks obtained by them in various courses. It has three tables:

- *Student_sample*(SRN, name) – stores information about students which includes their name and SRN (a unique identifier for each student).
- *Marks_sample*(SRN, Course, Marks) - stores information about marks obtained by students(identified by their SRN) in different courses.
- *Marks_history*(SRN, Course, Marks) – a backup table that stores information that has been deleted from the *Marks_sample* table.

The entries in the tables are shown below:

Student_sample:

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Note that the Marks_history table is initially empty.

Example: Create a trigger that checks if the entered marks are valid or not whenever an entry is made in the Marks_sample table.

Code:

```
DELIMITER //  
  
CREATE TRIGGER CheckMarks  
BEFORE INSERT ON Marks_sample  
FOR EACH ROW  
BEGIN  
    IF NEW.marks < 0 OR NEW.marks > 100 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Invalid marks: Marks must be between 0 and 100';  
    END IF;  
END;//  
  
DELIMITER ;
```

- The given SQL statement creates a trigger *CheckMarks*. It gets executed **whenever there is an insertion** in the *Marks_sample* relation. The keyword BEFORE specifies that the **trigger execution occurs before the triggering operation** (in this case, insertion of tuples in the *Marks_sample* relation) is executed.
- The trigger action gets executed **for every row that is inserted**. This is specified by the FOR EACH ROW clause.
- The trigger checks if the *Marks* value of the inserted tuple is **lesser than 0 or greater than 100** (i.e. it checks if the **Marks value is invalid**). If the specified condition is satisfied, the trigger raises an **error**, an appropriate **error message** is displayed and the **tuple is not inserted** (trigger execution occurs before insertion). If the condition is not satisfied, the tuple is inserted.

Let's try inserting into the *Marks_sample* relation using the statement:

```
INSERT INTO Marks_sample VALUES (1,'C03',101);
```

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Output:



13 12:19:02 INSERT INTO Marks_sample VALUES (1,'C03',101)

Error Code: 1644. Invalid marks: Marks must be between 0 and 100

0.000 sec

The tuple does not get inserted as the *Marks* value is greater than 100.

Note that if the *Marks* value is NULL, then the tuple gets inserted.

```
INSERT INTO Marks_sample VALUES (1,'C03',NULL);
```

BEFORE:

SRN	Course	Marks
1	C01	98
1	C02	89
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

AFTER:

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Example: Implement a trigger that mimics the behaviour of ON UPDATE CASCADE.

Code:

```
DELIMITER //
CREATE TRIGGER update_marks_on_cascade
BEFORE UPDATE ON Student_sample
FOR EACH ROW
BEGIN
    SET FOREIGN_KEY_CHECKS = 0;
    UPDATE Marks_sample SET SRN = NEW.SRN WHERE SRN = OLD.SRN;
    SET FOREIGN_KEY_CHECKS = 1;
END;//
DELIMITER ;
```

NOTE : SET FOREIGN_KEY_CHECKS = 0 disables all foreign key checks(i.e. even if a foreign key constraint violation occurs, an error will not be raised). This is done temporarily to update the *Marks_sample* relation, after which the foreign key checks is enabled by specifying the SET FOREIGN_KEY_CHECKS = 1

- The trigger *update_marks_on_cascade* updates the *SRN* values of tuples in the *Marks_sample* relation which are related to the tuples(in the *Student_sample* relation) being modified.
- To see its effect, let's execute the given query:

UPDATE Student_sample SET SRN = 5 WHERE name = 'Harry';

BEFORE:

AFTER:

Student_sample

SRN	Name
1	Harry
2	Ron
3	Hermione
4	Draco

Marks_sample

SRN	Course	Marks
1	C01	98
1	C02	89
1	C03	NULL
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98

Student_sample

SRN	Name
2	Ron
3	Hermione
4	Draco
5	Harry

Marks_sample

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

- The trigger automatically **updates the SRN values of all tuples** in the *Marks_sample* relation that are **related** to the tuple with *name* 'Harry'.

If this trigger would not have been created (and if ON UPDATE CASCADE was not specified in the foreign key constraint in the *Marks_sample* table), then:

- updating the SRN of any tuple in the *Student_sample* relation that have **related tuples** in the *Marks_sample* relation would have **thrown an error** as this updation will violate the foreign key constraint.
- However, updating the SRN of tuples that do not have any related tuples in the *Marks_sample* relation would not thrown an error.

Example: Implement a trigger that stores all tuples deleted from the *Marks_sample* table in the *Marks_history* table

Code:

```
DELIMITER //
CREATE TRIGGER backup_marks_info
BEFORE DELETE
ON Marks_sample
FOR EACH ROW
BEGIN
INSERT INTO Marks_history SELECT * FROM Marks_sample WHERE SRN = OLD.SRN AND COURSE =OLD.COURSE;
END //
DELIMITER ;
```

Let's see the effect of this trigger by executing the given query:

```
DELETE FROM Marks_sample WHERE SRN = 2;
```

BEFORE:

AFTER:

Marks_sample

Marks_history

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

(empty set)

SRN	Course	Marks
-----	--------	-------

Marks_sample

Marks_history

SRN	Course	Marks
3	C01	95
3	C02	98
5	C01	98
5	C02	89
5	C03	NULL

SRN	Course	Marks
2	C01	100
2	C02	98
2	C03	80



THANK YOU

Dr. Nagasundari S

Department of Computer Science and Engineering

nagasundaris@pes.edu